

Auto Loader vertieft: Schema Enforcement, Schema Evolution und Erkennungsmodi

Section 2 · Data Ingestion and Loading · Quelle: Databricks-Dokumentation (Auto Loader)

Ein früheres Kapitel dieser Section hat Auto Loader bereits im Zusammenspiel mit Streaming Tables eingeführt. Für die Prüfung sind darüber hinaus drei Details entscheidend: wie Auto Loader neue Dateien überhaupt *erkennt* (zwei unterschiedliche Betriebsmodi), wie er auf *Schema-Abweichungen* reagiert (Enforcement vs. Evolution), und wie sich das Verhalten gezielt konfigurieren lässt.

1. Zwei Erkennungsmodi: Directory Listing vs. File Notification

Auto Loader muss bei jedem Lauf herausfinden, welche Dateien im Quellverzeichnis neu hinzugekommen sind. Dafür stehen zwei Modi zur Verfügung:

- **Directory Listing** (Standard) – Auto Loader listet das Quellverzeichnis inkrementell auf und vergleicht das Ergebnis mit einem intern geführten Zustand bereits verarbeiteter Dateien. Vorteil: funktioniert ohne jede Zusatzkonfiguration. Nachteil: bei sehr vielen Dateien/Unterverzeichnissen wird das Auflisten selbst zum Flaschenhals.
- **File Notification** – Auto Loader abonniert stattdessen Ereignis-Benachrichtigungen des Cloud-Speichers (z. B. AWS S3-Events über SQS, Azure Event Grid mit Queue, GCP Pub/Sub) und erfährt so sofort und ohne Auflisten, wenn eine neue Datei ankommt. Das ist schneller und günstiger bei sehr großen, tief verschachtelten oder häufig wachsenden Verzeichnissen, erfordert aber die Einrichtung der entsprechenden Cloud-Infrastruktur (Event-Quelle, Warteschlange).

```
(spark.readStream
  .format("cloudFiles")
  .option("cloudFiles.format", "json")
  .option("cloudFiles.useNotifications", "true")    # File-Notification-Modus aktivieren
  .option("cloudFiles.queueUrl", "<sqs-queue-url>")  # nur bei AWS/SQS nötig
  .load("/Volumes/catalog/schema/raw_events"))
```

2. Schema Enforcement: Abweichungen werden nicht stillschweigend ignoriert

Sobald Auto Loader ein Schema kennt (aus einem inferierten oder explizit angegebenen Schema), wird jede eingehende Datei dagegen geprüft. Enthält eine Datei unerwartete zusätzliche Spalten, landen diese – statt verworfen zu werden – automatisch in der `_rescued_data`-Spalte als JSON-String. Damit geht bei einer Schema-Abweichung nie Information verloren, ohne dass der Ingest-Prozess deswegen abbricht.

3. Schema Evolution: Wie Auto Loader auf neue Spalten reagiert

Über die Option `cloudFiles.schemaEvolutionMode` lässt sich steuern, was passieren soll, wenn Auto Loader eine neue Spalte entdeckt, die im bisherigen Schema noch nicht vorkam:

Modus	Verhalten bei neuer Spalte
<code>addNewColumns</code> (Standard)	Der Stream wird gestoppt, das Schema um die neue Spalte erweitert, der Stream muss neu gestartet werden.
<code>rescue</code>	Neue Spalten werden nicht ins Schema aufgenommen, sondern landen dauerhaft in <code>_rescued_data</code> .
<code>failOnNewColumns</code>	Der Stream schlägt fehl und muss mit einem manuell aktualisierten Schema neu gestartet werden.
<code>none</code>	Neue Spalten werden ignoriert (landen nicht einmal in <code>_rescued_data</code>) – nur sinnvoll, wenn ein explizites Schema samt <code>schemaHints</code> vorgegeben wird.

```
(spark.readStream
  .format("cloudFiles")
  .option("cloudFiles.format", "csv")
  .option("cloudFiles.schemaLocation", "/Volumes/catalog/schema/_schema/orders")
  .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
  .option("cloudFiles.schemaHints", "order_id INT, amount DOUBLE")
  .load("/Volumes/catalog/schema/raw/orders"))
```

Die Option `cloudFiles.schemaLocation` ist dabei Pflicht, sobald Auto Loader das Schema selbst ableiten oder weiterentwickeln soll – dort wird das aktuell erkannte Schema zwischen den Läufen persistiert.

Ergänzt aus der offiziellen Databricks-Dokumentation: Databricks empfiehlt File-Notification-Modus für die meisten produktiven Workloads mit großen, wachsenden Datenmengen, da er schneller und kosteneffizienter skaliert als Directory Listing – Directory Listing bleibt aber der Standard, da er ohne jede Cloud-Infrastruktur-Einrichtung sofort funktioniert. Zusätzlich unterstützt Auto Loader seit neuerem automatische Typ-Erweiterung (`addNewColumnsWithTypeWidening`): Ändert sich z. B. eine Spalte von `int` zu `long` , wird der Datentyp automatisch erweitert, ohne dass bestehende Daten neu geschrieben werden müssen.

Quelle: [Configure schema inference and evolution in Auto Loader – Databricks-Dokumentation](#)